PRODUCT BLUEPRINT · V1.0

재즈 즉흥 악보 생성 앱 기획안 및 확장 설계도

코드 진행 위에 매번 새로운 연습용 즉흥 솔로 악보를 만들어 주는 도구.
규칙 기반 프로토타입(v0.3)을 기반으로, 머신러닝·딥러닝·RAG로 확장하는 전체 로드맵과
시스템 설계, 데이터 전략, 비용, 재시작 지침을 담은 마스터 문서.

현황 v0.3 작동 프로토타입 완성 (관악기 전용·음역군·3연음·B♭ 기보) · **작성일** 2026년 10월 12일 · **문서 용도** 프로젝트 재시작·외부 공유용 마스터 기획안

목차

1. 프로젝트 개요 — 무엇을 만드는가 3

2. 현재 구현 상태 — v0.3 프로토타입 명세 5

3. 제품 로드맵 — v1에서 v4까지 7

4. ML·DL·RAG 확장 설계 8

4.1 데이터 전략 8

4.2 생성 모델 진화 경로 8

4.3 RAG — 릭 검색·주입 설계 8

4.4 개인화 구조 9

4.5 평가 프레임 9

5. 시스템 아키텍처 10

6. 기술 스택과 학습 로드맵 11

7. 법적·사업 프레임 12

8. 비용·일정 요약 13

부록 A. v0.3 엔진 파라미터 14

부록 C. 새 세션 시작 프롬프트 16

> **이 문서의 용도.** 언제든 새로운 대화·새로운 개발자·새로운 도구로 프로젝트를 재개하기 위한 기준점(single source of truth)입니다. 특히 부록 C의 시작 프롬프트를 새 대화에 붙여넣으면 v0.3의 설계 결정을 모두 이어받아 작업할 수 있습니다.

1. 프로젝트 개요 — 무엇을 만드는가

1.1 한 줄 정의

코드 진행을 고르면 "어울리는 음"만으로 구성된 연습용 즉흥 솔로 악보를 매번 새로 만들어 주고, 반주와 함께 들려주는 앱. 악보 없이 즉흥연주를 하고 싶지만 좌절한 연주자들 — 설립자 본인 포함 — 을 위한 연습 도구다.

1.2 문제 정의

- 재즈 즉흥연주 학습의 벽은 "이론(코드·스케일)은 아는데 실제로 솔로를 만들지 못하는" 간극이다. 교재는 정적인 예시 릭만 준다.
- 업계 표준인 iReal Pro(유료, 50만+ 다운로드, 평점 4.7)는 *반주*는 해주지만 솔로를 만들어 주지는 않는다. 생성형 즉흥 악보의 자리는 사실상 비어 있다.
- 유일한 선례인 학술 도구 Impro-Visor(2006~)는 문법 기반 솔로 생성의 기술적 가능성을 20년 전에 증명했으나, 데스크톱 학술 소프트웨어로 머물러 마지막 버전이 2019년에 나왔다.

1.3 해결 원리

음을 네 계층으로 분류해 확률적으로 선택한다 — **코드톤 → 컬러톤(스케일음) → 어프로치음 → 인클로저·비밥**. 리듬은 3연음 지원 그리드에서 난이도별 어휘(4분음표 → 8분음표·엇박 진입 → 3연음·당김음)로 확장된다. 악기(금관·목관)별 음역·도약·호흡 제약을 강제한다. 이 규칙 엔진이 지금 작동 중이며(v0.3), 머신러닝은 이 위에 얹는다 — 규칙을 대체하는 것이 아니라 보강한다.

1.4 타깃 사용자

세그먼트	특징	우선순위
클래식 출신 실용음악 전공자·직장인 연주자	이론은 있으나 즉흥 어휘가 부족해 좌절한 층. 본인 포함 주변에 다수 존재	1순위 — 초기 베타·입소스
음대·실용음악과 학생	시험·앙상블 과제에서 즉흥 능력 필요. 교육기관 B2B 경로	2순위 — 구독 매출 코어
취미 재즈 연주자(국내·일본·미국)	꾸준한 구독 수요. iReal Pro 구매 이력자	3순위 — 확장 시장

1.5 차별화 포인트

- **매번 달라지는 악보** — 정적 릭 라이브러리가 아니라 즉석 생성.
- **연주 가능성 보장** — 악기별 음역·도약·호흡 제약을 위반하는 악보가 나오지 않음(엔진이 강제).
- **난이도 사다리** — "포기자가 다시 시작하는" 커리큘럼 설계. 기술이 아니라 교육 설계가 진입장벽.
- **취향 학습** — 마음에 든 솔로(♡)를 저장하면 점점 사용자 스타일에 맞게 생성(4.4절).

2. 현재 구현 상태 — v0.3 프로토타입 명세

2.1 완성된 것 (2026. 10. 기준)

영역	구현 내용	검증
생성 엔진	4계층 확률 선택(코드톤/스케일음/어프로치음/인클로저) + 비밥 스케일 + 하모닉 마이너(b9·b13)·하프홀리(♯9) 스케일 자동 적용	900개 조합 전수 테스트
리듬 엔진	틱 그리드(1틱=8분음표 1/3). 난이도별 어휘: 입문(4분·2분) → 중급(8분·엇박 진입·쉼표) → 심화(3연음·당김음)	마디 완결 100%
악기 제약	금관·목관: B3 이하 최소화, 8마디 프레이즈 내 5도 초과 도약 차단, 2마디마다 호흡 쉼표(목관)	최대 도약 5도 이하 (잔여 예외 0.3%)
음역군	A군 C4–B4 / B군 C5–B♭5. 4마디 유지 → 2마디 전환 창(이전 군 다리음 허용) → 8마디 프레이즈	군 혼재 위반 0건
곡 DB	Autumn Leaves(Gm, 24마디 풀폼, E♭7→D7♯9 변형 포함)·All of Me(C, 32마디) — 리얼북 차트의 *코드 진행만* 수록, 원곡 멜로디 미포함. 연습용 진행 3종(Ⅱ-Ⅴ-Ⅰ·블루스·마이너 Ⅱ-Ⅴ)	사용자 첨부 에튜드 6장 기반
출력	ABC → 오선보 렌더링(abcjs), 코드 심볼, 3연음 기호. B♭ 악기용 기보 전환(테너 색소폰)	스크린샷 시각 검사
재생	Web Audio 신디사이저 — 스윙 8분음표 반주(피아노·베이스·라이드) + 카운트인 + 마디 하이라이트	동작 확인
저작 보호 설계	생성 결과는 새 선율. 곡은 코드 진행만 반영. 유명 연주자 녹음 학습은 배제(법적 사유, 7장)	—

2.2 의도적으로 뺀 것 (다음 버전 과제)

- 마디선을 넘는 붙임줄 당김음(tie) 표기 — 현재는 마디 안 당김 리듬만 어휘화.
- 음역군 전환 시 도약의 완전 자동 관리 — 심화·목관에서 6도 도약 0.3% 잔여.
- 기타·건반(자유) 모드 UI 노출 — 엔진에는 존재하나 관악기 중심 정책에 따라 숨김.
- 계정·영구 저장·곡 검색 — 웹 MVP(v1) 범위.

2.3 기술 구성 (파일 하나)

프로토타입은 `index.html` 한 파일로 완결된다: ① 순수 로직 엔진(음악 이론·생성·ABC 변환, UI와 분리 — Node로 단위 테스트 가능) ② UI 컨트롤 ③ Web Audio 재생부 ④ abcjs CDN 렌더링. 엔진은 시드 기반 난수(`mulberry32`)라 같은 시드는 같은 악보를 만들어 **재현 가능한 테스트**가 가능하다. 이 "엔진/프레젠테이션 분리"는 v1 이후에도 유지하는 핵심 설계 원칙이다.

3. 제품 로드맵 — v1에서 v4까지

버전	목표	핵심 내용	기간(누적)	성공 지표
v1 웹 MVP	몇 명이 쓰는가 확인	계정·영구 저장(♡), 곡 DB 확장(10곡+), 곡 검색 UI, 3연음 예외 패치, 모바일 반응형, PDF 내보내기	+1~3개월	주간 생성 악보 수, 저장률, 7일 재방문율
v2 앱화·과금	지불 검증	Flutter 앱(안드로이드+iOS), 구독/코스 과금, 난이도 커리큘럼(L1~L5) 상품화, 교육기관 할인	+2~4개월	유료 전환율, 월 구독 이탈률
v3 ML 고도화	품질·개인화 도약	공개 데이터셋 학습 조건부 생성 모델, 릭 벡터DB(RAG), 사용자 취향 모델 — 4장 참조	+3~6개월	블라인드 인간 평가 승률, 저장률 상승폭
v4 플랫폼	학습 시장	사용자 MusicXML 업로드→스타일 학습, 연주 오디오 평가(마이크 인식), 교사용 대시보드	이후	생성-연주-피드백 루프 완성도

> **원칙.** ML(v3)은 규칙 엔진(v0~v1)의 사용자 데이터가 모인 뒤에 시작한다. 규칙 엔진은 ① 초기 품질 보증 ② ML 모델의 학습 신호(저장/거부) 수집기 ③ ML 실패 시 폴백 — 세 가지 역할을 계속한다.

4. ML·DL·RAG 확장 설계

4.1 데이터 전략

계층	데이터	용도	라이선스 상태
공개 연구 자원	Weimar Jazz Database — 수작업 전사 재즈 셀로 약 456개, 코드·연주자·리듬 메타데이터 포함	문법 학습·생성 모델 사전학습·평가 벤치마크	연구 공개 — **상업 사용 조건은 사용 전 확인 필수**
자체 말뭉치	곡 DB의 코드 진행(저작권 밖), v0 엔진이 생성한 대량 솔로(시드+파라미터 명시)	모델 예열(합성 데이터), 규칙 준수 습득	자체 확보 — 안전
사용자 데이터	♡ 저장·거부·템포·악기·연습 시간, (v4) 연주 오디오	개인화·평가 신호	약관 동의 기반 — 7장
배제 데이터	원곡 멜로디, 유명 연주자 녹음·전사본	—	법적 리스크로 사용하지 않음

4.2 생성 모델 진화 경로

1. **0단계(현재): 확률적 규칙 엔진.** 4계층 가중치 + 리듬 템플릿. 결정적이고 설명 가능하며 제약 준수가 보장된다.
2. **1단계: 문법 학습(가벼운 ML).** Weimar 데이터에서 릭 문법을 학습해 v0 문법 파라미터를 갱신한다(Impro-Visor의 "Learning Jazz Grammars" 접근의 현대적 재현). 통계적 프레이즈 분포가 실제 연주자 분포와 비슷해진다. 소규모 데이터로도 가능하다.
3. **2단계: 조건부 Transformer(딥러닝).** 코드 진행·난이도·악기·음역군을 조건으로 주면 솔로 토큰 열을 생성하는 인과 언어모델. 선행 연구로 Jazz Transformer(리드시트 생성), Anticipatory Music Transformer(제어형 인필링)가 경로를 검증했다. 합성 데이터(v0 엔진 산출)로 예열 후 실데이터로 미세조정. *핵심 아키텍처 결정: 생성은 ML이 하되, 후처리 검증기로 v0 규칙(음역·도약·군·마디 완결)을 항상 통과시킨다* — "ML 생성 + 규칙 검증" 하이브리드.
4. **3단계: 개인 스타일 어댑터.** 개인별 저장 솔로를 few-shot 조건 또는 경량 미세조정(LoRA급)으로 반영. 4.4절과 연결.

4.3 RAG — 릭 검색·주입 설계

RAG(검색 증강 생성)의 역할은 "모델이 기억하지 못하는 좋은 프레이즈를, 필요한 화성 문맥에서 꺼내 주는 것"이다.

구성 요소	내용
인덱스 대상	공개 전사본과 사용자 저장 솔로에서 추출한 **프레이즈 단위 조각**(1~2마디 릭). 각 조각의 메타데이터: 코드 문맥(진행상 위치·코드 종류), 난이도, 악기 적합성, 리듬 피처(틱 패턴)
임베딩	음악 전용 임베딩(예: 피아노롤·리듬 벡터의 학습형 임베딩) 또는 일반 임베더에 릭을 텍스트화(음정 등차·리듬 비율 서술)해 색인. 시작은 후자 — 인프라가 단순
쿼리	생성 시점의 상황: "II-V-I 로 진입 직전, A군, 난이도 2, 사용자가 최근 저장한 스타일"
주입 방식	① 상위 k개 릭을 생성 프롬프트의 예시로 제공(Transformer 단계) ② 규칙 엔진 단계에서는 검색 릭의 리듬 패턴을 템플릿 후보에 추가
안전장치	검색 조각이 특정 곡의 원곡 멜로디와 지나치게 유사하면(n-gram 유사도 문턱) 인덱스에서 제외 — 원곡 멜로디 오염 방지

4.4 개인화 구조

- **즉시 반영(v1):** ♡ 저장 시 그 솔로의 리듬 어휘·계층 가중치 편차를 사용자 프로필에 누적 — 다음 생성의 사전확률을 조정.
- **선호 모델(v3):** 저장/거부를 보상 신호로 하는 경량 학습(밴딧 계열)으로 계층 가중치·리듬 분포를 개인화.
- **스타일 조건(v3):** 저장 솔로 상위 N개를 생성 프롬프트의 in-context 예시로 사용(few-shot 개인화) — 모델 재학습 없이 취향이 반영된다.

4.5 평가 프레임

구분	지표	합격선(초안)
자동·음악이론	마디 완결율, 음역·도약·음역군 준수율, 코드톤 강박 비율, 리듬 다양성(엔트로피), 원곡 유사도(n-gram) 역검사	준수율 99.5% 이상 / 유사도 문턱 미달
자동·스타일	Weimar 실제 셀로와의 음정·리듬 분포 거리	v0 대비 개선
인간	연주자 블라인드 A/B(v0 규칙 vs ML 하이브리드) — "어느 쪽이 사람 같은가" + 연주 가능성 평가	ML 하이브리드 60% 이상 승률에서 전환
제품	저장률·7일 재방문(3장 지표와 동일)	절대치는 v1 베타에서 확정

5. 시스템 아키텍처

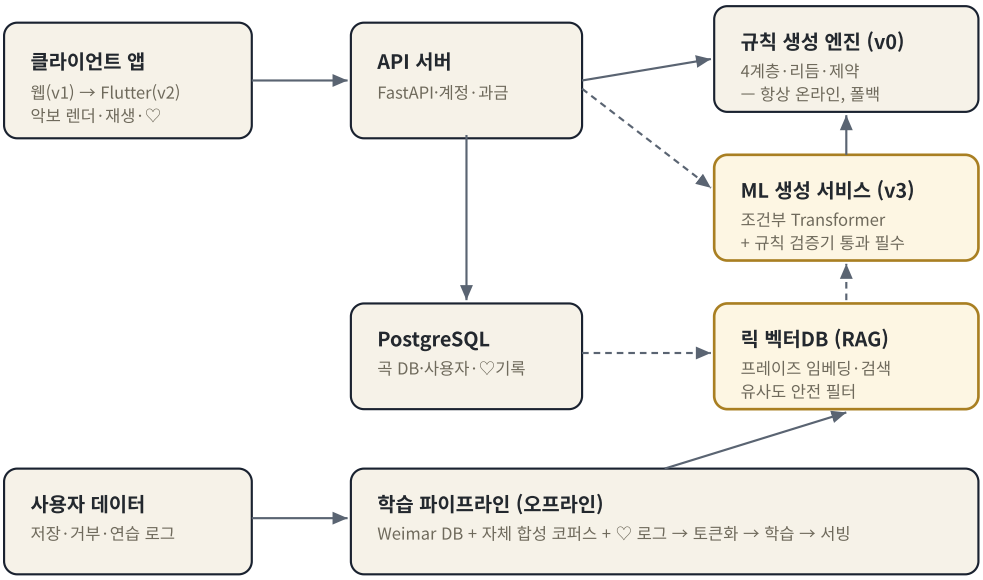


그림 1 — 전체 아키텍처. 실선 = 생성 요청 경로, 점선 = v3 ML·RAG 경로. 규칙 엔진은 v3 이후에도 검증기·폴백으로 상시 유지된다.

5.1 데이터 흐름 (생성 요청 1회)

1. 클라이언트: 곡/진행 · 조 · 난이도 · 악기 · 기보(B♭/콘서트) 선택 → API 요청.
2. API: 사용자 프로필(가중치 편차) 조회 → 생성 서비스 호출. v3에서는 릭 벡터DB 검색 결과를 조건에 추가.
3. 생성: 규칙 엔진 또는 ML 모델이 솔로 생성 → **규칙 검증기**(마디 완결·음역·도약·군·유사도) 통과. 실패 시 재시도 후 규칙 엔진 폴백.
4. 응답: 솔로 데이터(틱 이벤트) + ABC 표기 + 메타(사용 기법 태그) → 클라이언트가 악보 렌더링·반주 재생.
5. 피드백: ♡/거부 → PostgreSQL 기록 → 개인화·학습 데이터로 축적.

6. 기술 스택과 학습 로드맵

6.1 스택 (단계별)

계층	v1 (현재→MVP)	v3 (ML)
생성 엔진	JavaScript(브라우저·Node 공유) — v0.3 그대로	Python + PyTorch 조건부 Transformer, 검증기는 v0 규칙 이식
음악 이론·처리	자체 구현(v0) → 필요 시 music21(파이썬)로 확장	music21·MIDI/MusicXML 파서, 토크나이저(REMI류)
프론트	단일 HTML + abcjs (v0 그대로 → 계정 기능만 추가)	동일 + 개인화 UI
앱	—	Flutter(안드로이드+iOS 동시) — v2부터
백엔드	FastAPI + PostgreSQL + 무료 티어 클라우드	+ 벡터DB(pgvector 시작 → 전용 전환), GPU 추론 서버
RAG	—	pgvector 임베딩 색인 → 검색 주입 파이프라인

6.2 코딩 초보자의 학습 로드맵 (AI 페어링 전제)

1. **지금(v0):** 코딩 학습 선행 불필요 — AI가 전부 작성·검증하며, 사용자는 음악적 판단·테스트만. 단, "엔진/프레젠테이션 분리, 시드 기반 테스트" 같은 설계 어휘만 익혀두면 지시 품질이 올라간다.
2. **v1기:** 파이썬 기초(변수·함수·딕셔너리) — music21로 악보 읽기 놀이. GitHub 이슈로 요구사항 관리 습관.
3. **v3 진입 전:** 파이토치 튜토리얼 수준의 감각 + 음악 정보검색(MIR) 입문 문헌. 실제 구현은 AI 페어링, 리뷰 기준은 4.5절 평가 프레임으로.

7. 법적·사업 프레임

항목	방침	근거
원곡 멜로디	**사용하지 않음** — 곡은 코드 진행만 수록	리얼북 원본은 불법 출판물이었고 2004년 할 레너드 6판이 첫 합법판. iReal도 'Real Book' 이름 분쟁으로 개명
코드 진행	사용 가능 (통설상 보호 대상 아님) — 상업화 전 변호사 확인 유지	iReal Pro 선례
생성 악보	새 선율. 원곡·유명 솔로와의 n-gram 유사도 역검사 상시 적용	4.5절
공개 데이터셋	상업 사용 조건 사전 확인 후 투입. 불명확하면 합성 코퍼스로 대체	Weimar 등 연구용 라이선스
사용자 데이터	개인화·모델 학습 동의 분리 수집, 옵트아웃 제공	개인정보보호법 준수
과금	부분 무료(일일 생성 한도) + 월 구독 무제한·코스 구매. 교육기관 라이선스 별도	iReal Pro 일회성 모델과 차별화되나, 코스 상품으로 일회성 선택지 병행

8. 비용·일정 요약

항목	금액	시점	비고
개발 (기획·코딩·디자인)	0원	전 단계	AI 페어링 개발
호스팅·백엔드	월 0 → 3만원대	v1~	무료 티어 시작, 사용자 증가 시 유료
도메인	연 1.5~2만원	v1~	.com 기준
Google Play 개발자	$25 일회성	v2	공식 요금
Apple Developer	$99/년	v2	iOS 빌드용 Mac 또는 클라우드 맥 별도
GPU 학습·추론	월 수만~수십만원	v3~	모델 크기에 따라 — 소형 조건부 모델로 시작해 억제
법무 자문(상표·저작권)	수십만원대	v2 말	코드 진행·상표 확인
앱스토어 입점까지	**약 10~50만원**		개발비 0 — 대부분 계정·도메인·법무

8.1 다음 30일 체크리스트

1. v0.3을 연주자 5~10명에게 배포 → "어느 난이도에서 어색한가" 메모 수집.
2. 잔여 결함 패치: 심화-목관 6도 도약 3건·쉼표 철자 누수.
3. 곡 DB 확장 — 리얼북 빈출곡 5곡 추가 수록(사용자 악보 제공).
4. 난이도 커리큘럼 L1~L5 기준표 초안(사용자 작성, 감수는 AI 지원) — v1 코스 상품의 뼈대.
5. 앱 이름 후보 3개 + 상표 선행검색.

부록 A. v0.3 엔진 파라미터

A.1 음 선택 계층 가중치

계층	강박(on-beat)	여백(off-beat)	비고
코드톤	+7	+2	인접 도약 선호 +7−2×거리
스케일음	+0.4	+2	걸음 흐름 담당
어프로치음	다음 코드 3음·7음 반음 인접 시 +6(심화 +9)		중급부터
인클로저	마디 시작 확률 30%(심화) — 목표음 ±1 감싸기		도약 6반음 이내 제한

A.2 악기·음역군 제약

모드	음역	도약	기타
금관	B3(59)~B♭5(82)	5도 초과 차단(순수 5도 15% 잔존)	옥타브급 도약 사실상 배제
목관	동일	동일	2마디마다 호흡 쉼표(3연음 보존)
기타·건반	C3(48)~E6(88)	자유(큰 도약 감쇠만)	UI 비노출(보류)
음역군	A=C4−B4(60−71), B=C5−B♭5(72−83). 8마디 프레이즈 = 4마디 유지 + 2마디 전환 창(이전 군 다리음 허용) + 2마디 신군		

A.3 리듬 어휘 (틱 단위 — 1틱=8분음표 1/3, 반마디=12틱)

난이도	어휘
입문	[6,6] 4분×2 · [12] 2분
중급	8분 조합 · [−3,3,6] 엇박 진입 · [9,3] 4분+8분 · [3,9] 8분+점4분
심화	중급 전부 + 3연음 [4,4,4]·[2,2,2,6]·[6,2,2,2] — 쉼표로 그룹이 깨지면 자동 재시도

A.4 검증 결과 (900개 조합 전수)

- 마디 완결·음역·음역군·3연음 무결성: **위반 0건**

- 제한 악기 최대 도약 9반음(6도) — 심화·목관 3건(0.3%) 잔여, 패치 예정
- 5도 도약 발생률 1.44% — "가급적 억제" 요구 충족
- 쉼표 해소가 도약 강제를 대체 — 프레이징 자연스러움 향상

부록 C. 새 세션 시작 프롬프트

새로운 대화에서 아래 블록을 그대로 붙여넣으면 프로젝트를 이어서 진행할 수 있다.

```
저는 '재즈 즉흥 악보 생성 앱' 프로젝트를 진행하고 있습니다.
마스터 기획안: jazz-solo-app-blueprint (이 문서의 복사본을 첨부하거나 요약을 공유)

현황 요약:
- v0.3 작동 프로토타입 완성: 단일 HTML 웹앱 (규칙 기반 생성 엔진)
- 엔진: 코드톤/스케일음/어프로치음/인클로저 4계층 확률 선택,
  틱 그리드(1틱=8분음표 1/3) 리듬 어휘(4분→8분·엇박→3연음·당김),
  금관·목관 제약(B3 이하 억제, 5도 초과 도약 차단, 목관은 2마디 호흡 쉼표),
  음역군 A(C4–B4)/B(C5–B♭5) 4마디 유지+2마디 전환 창,
  시드 기반 재현 가능, Node 단위 테스트 관습 유지
- 곡 DB: Autumn Leaves(Gm 24마디, E♭7→D7♯9 변형), All of Me(C 32마디) — 코드 진행만 수록
- 출력: ABC→abcjs 오선보, B♭ 악기용 기보 토글, Web Audio 스윙 반주 재생
- 검증: 900 조합 전수 테스트 (잔여 결함: 심화-목관 6도 도약 0.3%, 쉼표 철자 누수)

법적 원칙: 원곡 멜로디·유명 연주자 녹음 사용 금지. 곡은 코드 진행만.

이번 세션 과제: [여기에 구체적 작업을 적으세요 — 예: "잔여 결함 패치", "곡 5개 추가", "v1 계정·저장 구
현", "4.2절 1단계 문법 학습 프로토타입 시작"]
```

> **유지 관리 팁.** 프로젝트 진행 중 중요한 설계 결정이 바뀌면 이 문서의 해당 절을 갱신하고 새로 내보낸 PDF를 배포한다. 문서가 단일 기준점 역할을 하려면 결정이 대화에만 남지 않도록 하는 것이 핵심이다.